# Parallel Monte Carlo Criticality Calculation Using CUDA

**Zack Nguyen, Aaron Li, Aidan Wu**

## Problem

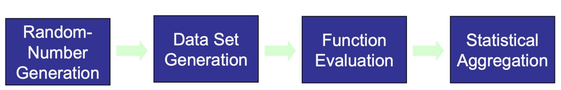
Nuclear fission is becoming ever more prevalent as an alternative energy source to fossile fuels to power large-scale infrastructures such as data centers, semiconductor fabrication plants, and other industrial facilities. To harness fission energy safely and efficiently, accurate and precise calculations are needed.

The goal is to construct a reactor that is stable and self-sustaining. This is measured by the effective multiplication factor, k_eff.

$$k_{eff} = \frac{\text{neutron production rate}}{\text{neutron loss rate}}$$

k < 1: subcritical (reactor dies out)
k = 1: steady state (critical)
k > 1: supercritical (neutron population grows, dangerous!)

**The Challenge:** Neutron transport equation in 2D/3D geometry with material heterogeneity requires billions of calculations
**The Solution:** Monte-Carlo is the most well-known stochastic method for simulating complex transport problems like neutron fission. Monte Carlo sampling of particle trajectories avoids discretization error




*Steps for Monte-Carlo method [3] used for neutron transport simulation to determine criticality*

We implement event-based Monte Carlo on NVIDIA A100 GPUs instead of history-based approaches. This is to achieve a higher occupancy through smaller, focused kernels for transport and collision events.

Cornell University

## Solution

Used energy distribution and cross sections from QualifyingMC

**Problems with Naive Parallel:**
- Neutrons have very different lifespans
  - One might die quickly, another might bounce around many times
- Leads to warp divergence & wasted GPU time (all threads are waiting on slowest neutron)

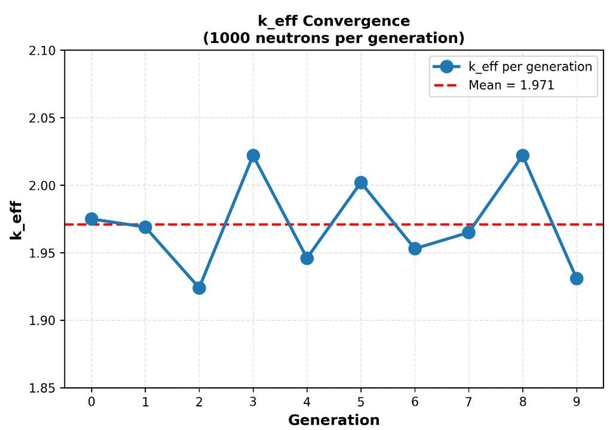
**Event-based algorithm: Cycle through stages using queues**
- Launch a kernel for each individual stage
- Kernel is a discrete step = less warp divergence

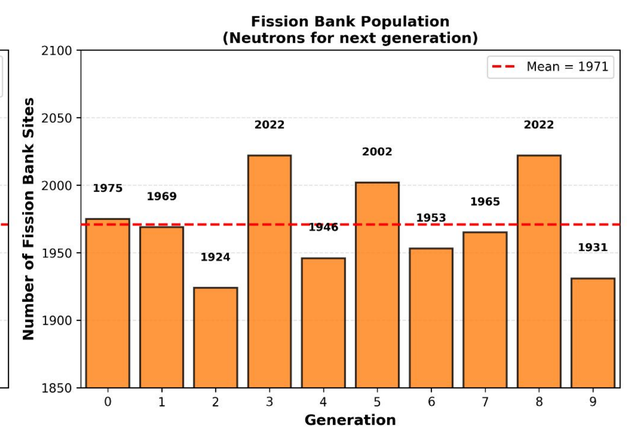
**Move stage:**
- Each neutron hits either a material boundary or a collision point
- If it hits material boundary: update position & collision probabilities of the neutron
- Otherwise, collide => add to collision queue

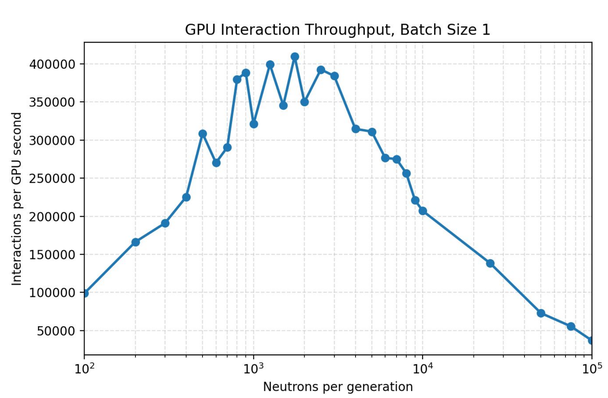
**Collision stage:**
- Neutron collision leads to 3 possibilities:
  - Scatter (neutron "bounces"): updates energy, send back to move queue
  - Capture (neutron is absorbed): kill the neutron
  - Fission: parent neutron dies, add child neutrons to fission bank

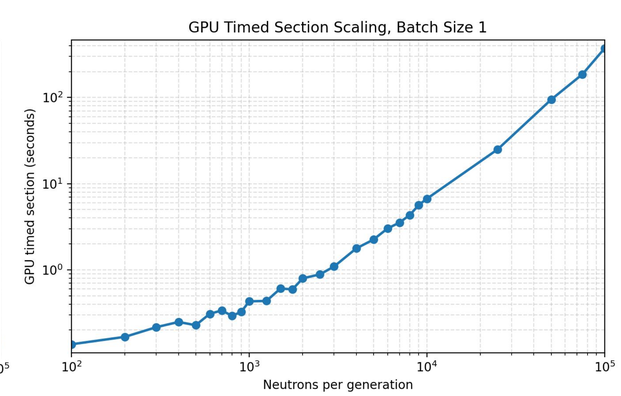
**Fission Bank:**
- To make the # of simulated neutrons consistent in each generation, we resample the next generation from the fission bank to mimic the distribution of produced neutrons

## Results



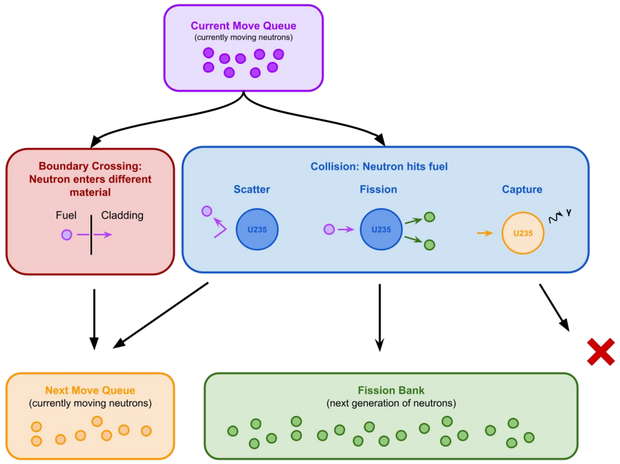




## Future Work

- Shannon Entropy as a mechanism for ending the sim
  - divide regions into cells and calculate entropy using counts per cell
  - converged entropy suggests converged fission distribution
- Current k_eff = 1.93 (supercritical, as expected). Phase 2 will implement absorping control rods and binary search to iteratively adjust rod insertion until k_eff = 1.0
- Benchmark against Hamilton and Evans work at Oak Ridge National Laborator [2] which uses the V100 Nvidia GPU
- Multi-generation convergence: Run 50+ generations to reduce statistical noise




## References

[1] Islam, ASM Fakhrul. Modeling Neutron Interaction Inside a 2D Reactor Using Monte Carlo Method. MS thesis. University of South Carolina, 2019.

[2] Hamilton, Steven P., and Thomas M. Evans. "Continuous-energy Monte Carlo neutron transport on GPUs in the Shift code." Annals of Nuclear Energy 128 (2019): 236-247.

[3] Sheppard, Andrew. CUDA-Accelerated Monte-Carlo for HPC ~ a Practitioner's Guide ~ F Ountainhead. Nvidia, Nov. 2011.

[4] Romano, Paul K., and Benoit Forget. "Parallel fission bank algorithms in Monte Carlo criticality calculations." Nuclear Science and Engineering 170.2 (2012): 125-135.